However, if your application architecture is edge compute dependent and pushes learned models to the edge sensors for faster responses, this (new) learning must be transferred regularly. Interestingly, in industrial implementation, edge computing utilisation is highly recommended.

The challenge is how to update these formulae in low-cost sensors. Especially when these sensors do not have large memory, or air firmware (OTA) updates are not feasible. How can you send and update only that basic formula to one sensor device, if needed?

If the solution is designed right, it would have accounted for such drift in the first place and would adapt to the changing scenarios. If that is not the case, either your solution would fail, or the performance will degrade. It may also mean lower accuracy, lower speed, high error margins, and other such issues.

However, remember that it is difficult to identify any scenario in which concept drift may occur, especially when you are training the model for the first time. So, instead of working on pre-deployment detection, your solution should safely assume that the drift will occur and accordingly have a provision to handle it.

Your solution cannot handle drifts; you will have to fix it eventually on an ongoing basis. It is a costly proposition as you keep spending money on constant fixes. The risk of tolerating poor performance for some time exists. This risk increases if the detection of drift is not apparent.

And if you fail to detect the drift for a more extended period, it can be quite risky on several fronts. Thus, the detection of drift is a critical factor. Check if your solution has accounted for concept drift and, if yes, then to what extent. The answer to that check would tell you the level of risk.

Some challenges in handling concept drift

The first and obvious challenge is to detect when the drift occurs. I recommend two possible methods to handle it:

1. When you finalise a deployment model, record its baseline performance parameters. After deployment, periodically monitor these parameters. If you see any difference, and if it is significant, it could indicate potential concept drift. In that situation, take action to fix it.
2. The other way to handle it is to assume drift will occur. It means you will put a plan in place to periodically update the cloud model and the sensors or edge network. The challenge, in that case, is to handle the edge sensor updates without significant downtime.

The first is easier to manage. However, the second poses a technical problem—so it becomes essential to have an in-built sensor capability that accepts model (formulae) updates without updating complete firmware. And this is where you must use the dynamic evaluation algorithm.

The dynamic evaluation algorithm

The basic algorithm was first introduced in 1954. It was first used in HP's desktop calculators in 1963. Now, almost all the calculators deploy this algorithm.

This algorithm relies on a specific type of representation of the formula, known as Reverse Polish Notation (RPN) or Postfix Notation.

The notation result is always context-free. It means once we convert an equation in Postfix Notation, it becomes easier for a computer to evaluate it. An outside-in evaluation sequence is used to perform this computation.

If you would like to learn more about this algorithm and how it can be implemented at a sensor level, please check *https://www.electronicsforu.com/electronics-projects/hardware-diy/calculator-using-postfix-notation.*

Other scenarios of concept drift

I explained concept drift with an example of an electrical motor. Nonetheless, the problem is not limited to this use case. Many other applications are vulnerable to this phenomenon and resulting problems.

For example, in credit card spend tracking and fraud detection algorithms used by banks consumer spending patterns change all the time.

For a security surveillance application used in public places, the visitor pattern can show seasonal or permanent change over time (see what happened with Covid-19). Advertising, retail, marketing, health applications are equally prone.

If your sensor network is monitoring server room temperature and humidity, a new cabinet or rack addition can also affect the pattern of change of these factors.

Summary

It is unrealistic to expect that data distributions stay stable over a long period. The perfect world assumptions in machine learning do not work in most cases due to changes in the data over time, which is a growing problem as these tools and methods increase.

The key to fixing it is acknowledging that AI or ML applications do not sit in a black box. These evolve continuously and must be supervised at all times. When the change occurs, we can regularly fix them in a near real-time environment with the proposed algorithm and technique. END

By: Anand Tamboli

The author is a serial entrepreneur, speaker, award-winning published author, and an emerging technology thought leader.

The article was originally published in the January 2021 issue of Electronics For You